Entrega 2

-Juan Sebastián Herrera Corzo 202515767-
-Nicolas Arce Sanabria 202513149-
-Juan Sebastian Leon 202510399-

Contexto del Problema

Este sistema cumple el rol de propuesta cuyo objetivo es centralizar y gestionar operaciones internas de un Board Game Café. La solución está diseñada para operar de forma únicamente local dentro del establecimiento, lo que implica que su funcionamiento principal no necesita de conexión a internet. El software soporta la interacción de tres actores fundamentales: los clientes, quienes gestionan sus mesas, pedidos y préstamos, los empleados, que están divididos entre meseros y empleados, pueden consultar y modificar sus turnos y el administrador, quien supervisa el inventario, finanzas y plantilla de empleados. El propósito del sistema es automatizar el control en tiempo real de personas que entran y salen del establecimiento, esto es en pocas palabras el control de aforo, aplicar reglas de compatibilidad en el préstamo de juegos físicos y fijar la facturación de la cafetería, garantizando la persistencia de datos que operan en el negocio.

Usos de Herencia y Polimorfismos

En el diseño de la aplicación aplicamos el patrón de herencia mediante clases abstractas en varios módulos, como usuarios, menú, préstamos y juegos de mesa. Por ejemplo, en lugar de crear clases gigantes y repetir código, lo cual es una mala práctica de programación, creamos la clase abstracta padre Usuario, que contiene datos comunes de login y password. De esta, heredan las clases de Cliente, Admin y Empleado. Esto permite mantener un código más limpio y legible, manteniendo únicamente atributos exclusivos para cada clase correspondiente, como puntos de fidelidad en clientes y Estado en turno en empleados.

Además, la herencia facilitó la implementación del polimorfismo, lo cual fue fundamental para simplificar la lógica de facturación. La muestra más precisa de esto está en la clase abstracta Venta y su método caclularTotal(). Al tener dos tipos de ventas (de juego y cafeteria), el sistema no necesita usar múltiples condicionales, sino solamente necesita llamar al método calcularTotal del objeto venta correspondiente. El polimorfismo permite saber al sistemas si debe aplicar 19% de IVA si es juego o el 8% si es cafeteria.

Controlador

Decidimos separar la lógica del negocio de los datos puros. Por esta razón, la arquitectura del modelo cuenta con la clase Cafe, que actúa únicamente como contenedor de datos (listas de

usuarios, mesas, etc). Toda la gestión del negocio, validaciones y cálculos es manejada por la clase CafeLogica. Esto garantiza que el modelo de datos sea más legible, entendible y tenga una estructura escalable a futuro.

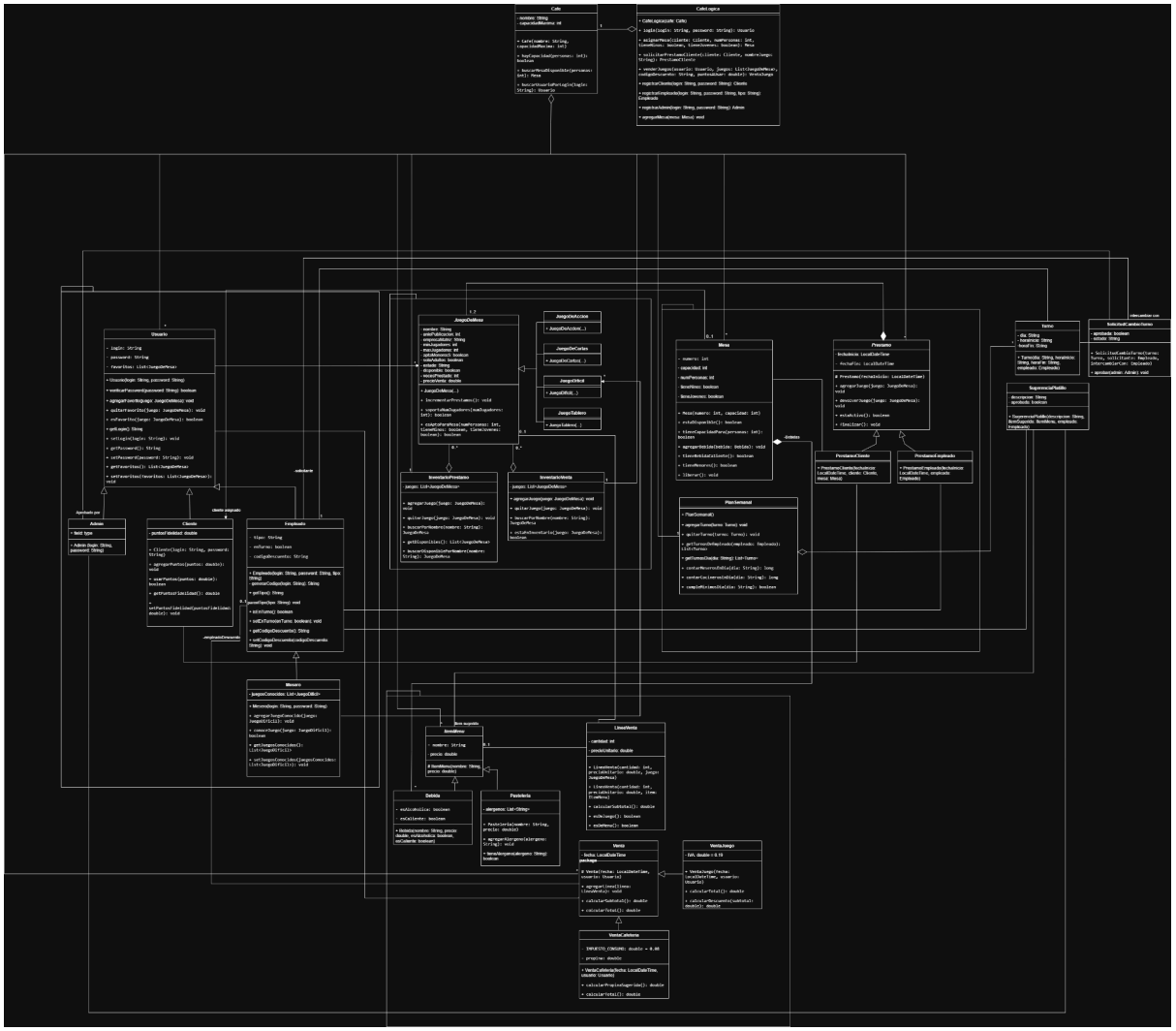


Figura 1. Modelo de clases detallado con atributos, métodos y multiplicidades

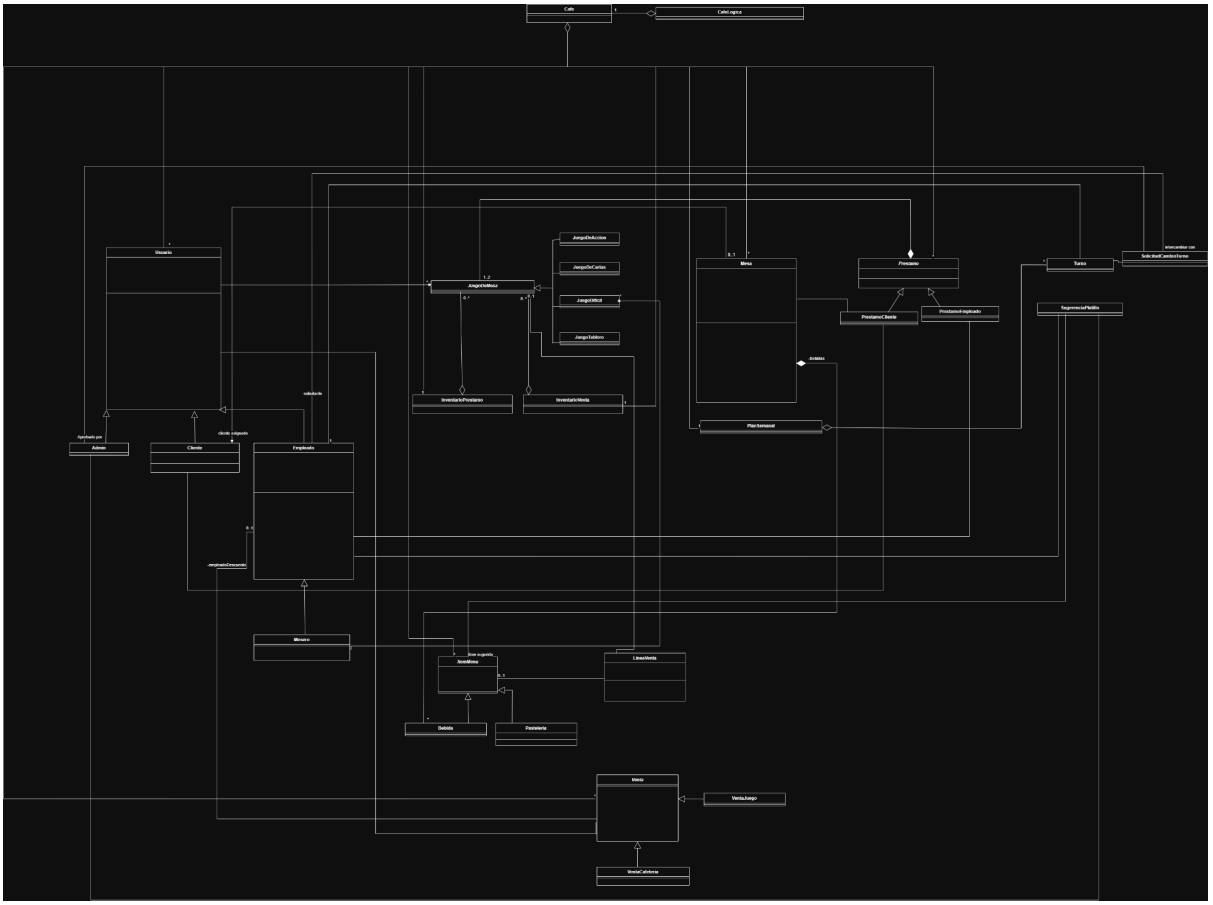
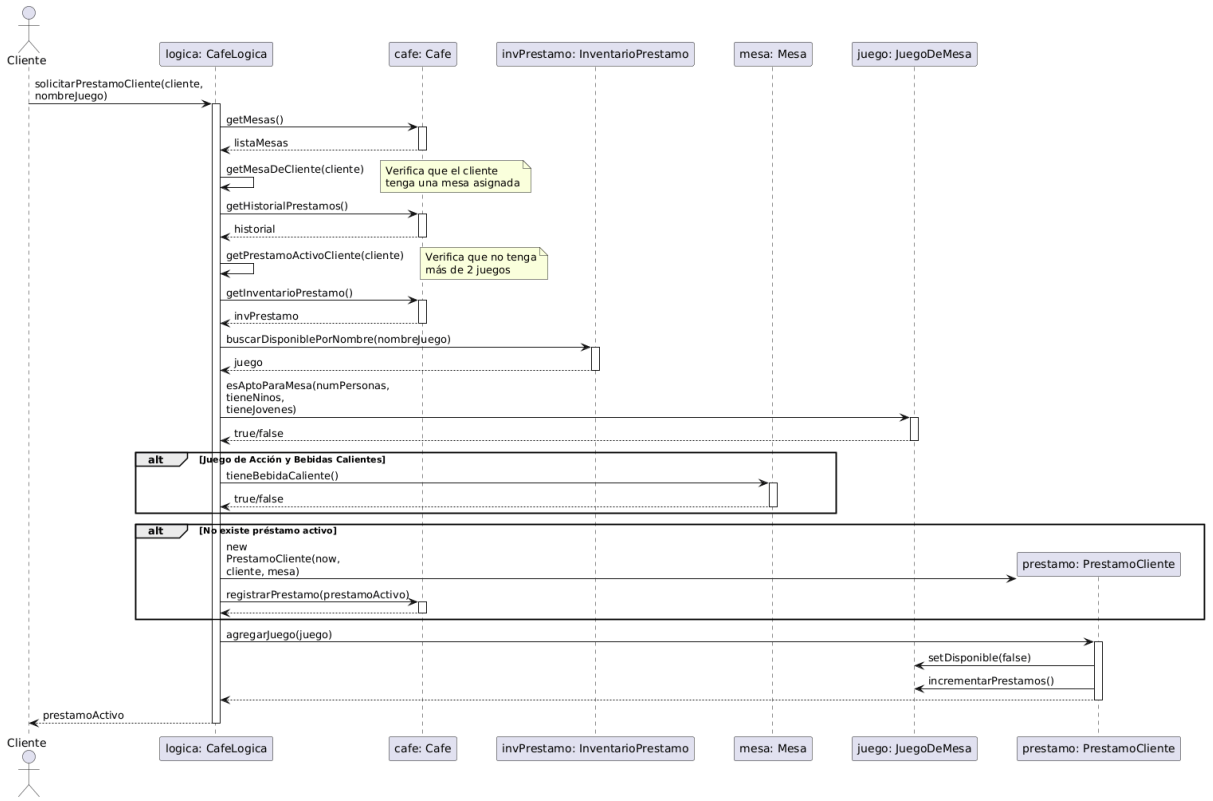
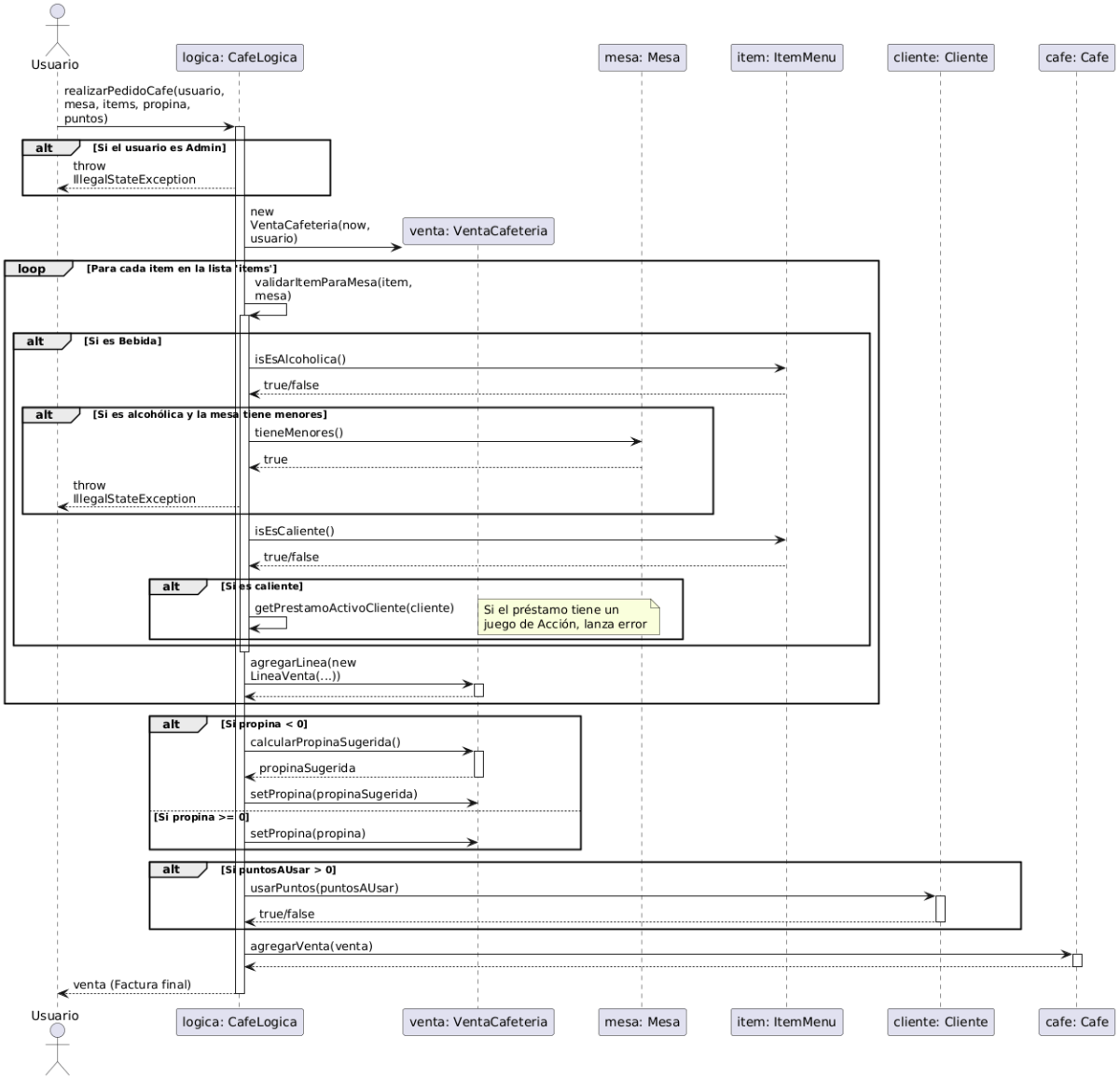


Figura 2. Modelo de alto nivel sin métodos ni atributos





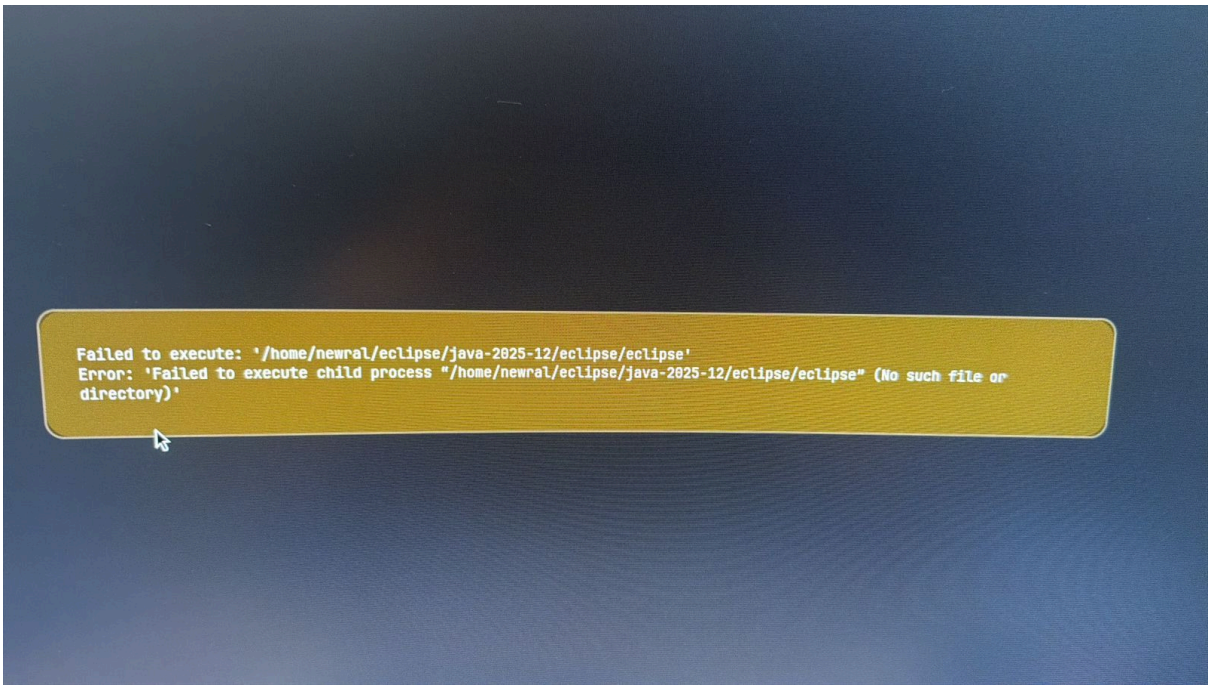
Los diagramas de secuencia nos permiten entender cómo funciona el código de una forma visual y dinámica. Las clases que aparecen en recuadros morados en la parte superior e inferior indican el momento en el que la función utiliza un método particular de esa clase, por ejemplo, new venta Cafetería señala la clase venta. Por otro lado, estas secuencias deben leerse de arriba a abajo, así se puede leer el flujo de la función de manera lógica. Además, los cuadros ALT indican que hay condicionales que validan que la función se pueda ejecutar o no.

USO DE IAG

Usamos IA para automatizar los getters y setters a la hora de desarrollar las clases de las entidades presentes en el paquete de modelo.

A su vez, la usamos para hacer el código más compacto en términos de líneas de código ya que la implementación fue más larga de lo esperada. Con esto buscamos tener un código más legible, más navegable y más claro al momento de entender. La IA, básicamente, nos recomienda hacer los métodos en una línea respetando los corchetes al momento de definir su contenido.

Antes de exponer las cosas puntuales con las cuales nos ayudó, decir que yo, Nicolas Arce, tuve un problema al momento de cambiar de la rama main a mi propia rama después de avanzar con la lógica. Uso lazy git, por lo que al ubicarme en el cuadro de las ramas y darle space a la mía, me pedía hacer un stash y luego un pop. Le di enter (ya que lo hace automáticamente) y, además de sobreescribir cosas sobre el código que venía desarrollando, me detono eclipse. Fuera de los errores de sintaxis por las sobreescrituras, me salian errores generales en el proyecto con un signo de exclamación rojo. Entendiendo que no era sintaxis, intenté hacerle track en problems pero de verdad que no encontré de donde venía el problema. Decidí reiniciar el IDE y cuando lo intente abrir de nuevo, me aparece esto:



Por lo que como último recurso llamé a claude code al proyecto y, ya habiendo explicado la situación, le di un prompt para que me ayudara a solucionar el problema (ya que quizás era algo que git modificó y yo genuinamente no me había dado cuenta). Lo intentó modificar pero sigue sin abrirme el IDE (la verdad es un problema).

Fuera de esta situación, usamos la IA para entender mejor cómo podemos manipular el tema de las fechas. Nos recomendó LocalDate y LocalDateTime, por lo que a partir de eso leímos su documentación respectiva.

Por último, nos recomendó usar System.out.println para aquellos casos donde no haya una excepción, pero al usuario se le deba indicar una información necesaria como los alérgenos en su comida como tip.